

Juan Sebastián Loaiza Ospina

B74200

Silvio Castillo Morales

C38910

Fecha: 14/11/2025

1) Investigación del problema

- a) El **problema de la Subsecuencia Común Más Larga (LCS)** consiste en encontrar la *secuencia de caracteres más larga* que aparece en **el mismo orden** en dos cadenas dadas, en este caso de ADN, aunque no necesariamente de forma continua.

Suponiendo que existen dos cadenas **ADN1** y **ADN2**, se busca encontrar una cadena **ADNx** que posea las características:

- 1) Estar formada únicamente por caracteres que aparecen tanto en **ADN1** como en **ADN2**.
- 2) Mantiene el mismo orden relativo en ambas cadenas.
- 3) Mayor longitud posible siguiendo las primeras dos reglas.

NOTA: Una **subsecuencia NO** es lo mismo que un **substring**. Los caracteres de la subsecuencia no tienen que aparecer seguidos uno tras otro, los caracteres de un hipotético **ADN2** pueden tener otros caracteres de por medio mientras conserven el orden de la cadena en **ADN1**.

2) ¿Por qué la fuerza bruta NO sirve como estrategia?

La solución por fuerza bruta consiste en:

- 1) Generar todas las subsecuencias posibles de **AND1**.
- 2) Para cada subsecuencia posible de **ADN1**, revisar si es una subsecuencia en **ADN2**.
- 3) Guardar la cadena más larga que cumpla los requisitos previos.

El problema que encontramos con esta técnica es que la cantidad de subsecuencias que podríamos encontrar en una hipotética cadena **ADN1** de longitud **N** es **2ⁿ**.

NOTA: La complejidad **temporal** de esto es **O(N · 2ⁿ)** porque para cada una de las **2ⁿ** subsecuencias generadas es necesario verificar si aparece dentro de **ADN2** y puede tomar hasta **O(N)** en el peor de los casos. En el caso de la complejidad **espacial** sería **O(N · 2ⁿ)** porque es la cantidad de caracteres en el peor de los casos y como mínimo habían **2ⁿ** secuencias almacenadas en memoria.

3) Algoritmo eficiente (programación dinámica para LCS)

Para usar el algoritmo de Programación Dinámica para LCS se construye una matriz donde cada posición indica la longitud de la mejor subsecuencia común entre los prefijos de las dos cadenas,

y la llena comparando caracteres y usando los resultados ya calculados, permitiendo obtener la subsecuencia más larga en tiempo $O(n \cdot m)$. Además, su estructura permite **paralelizar por diagonales**, lo cual es ideal para usar MPI que es requerido por el proyecto. Esto utiliza el método wavefront, que consiste en:

- a) Primero se calcula la diagonal 1 (independiente). /
 - b) Luego la diagonal 2 (que solo necesita la 1). //
 - c) Luego la diagonal 3 (que solo necesita la 2). ///
 - d) Y así sucesivamente con las demás diagonales. //
- 4) Pseudocódigo pensado para resolver el problema y posteriormente paralelizar con MPI

NOTA: S1 y S2 representan Substring 1 y Substring 2.

wavefrontLCS(variables S1, S2) {

 n = longitud(S1)

 m = longitud(S2)

 Crear matriz DP[n+1][m+1] llena de ceros

 // Recorrer la matriz por diagonales wavefront

 // Cada diagonal k cumple que: $i + j = k$

 // Esto permitirá en la versión paralela dividir cada diagonal entre varios procesos de MPI

 Para k desde 1 hasta (n + m - 1) hacer {

 // Cálculo del rango válido de i para esta diagonal

 i_min = max(1, k - m)

 i_max = min(n, k - 1)

 // Aquí, en paralelo, cada proceso podría tomar una parte del rango

 // por ejemplo: proceso 0 primeras celdas, proceso 1 las siguientes, etc.

 Para i desde i_min hasta i_max {

 j = k - i // porque deben sumar k

 Si (S1[i-1] == S2[j-1]) {

 // Si los caracteres son iguales, se extiende la subsecuencia

 DP[i][j] = DP[i-1][j-1] + 1

 Sino

 // Si son distintos, pasar el máximo del de arriba o el izquierdo

```

        DP[i][j] = max( DP[i-1][j], DP[i][j-1] )
    }

}

// En la versión MPI aquí hay que poner sincronización
// Algo tipo una barrera
// MPI_Barrier()??

}

Retornar cadena de caracteres
}

```

5) Segunda Version del algoritmo

La primera versión del algoritmo hace los cálculos en base de la diagonales, de la matriz, esta es eficiente pues usa la menor cantidad de procesos posibles. Este metodo puede que sea complicado de implementar como paso de mensaje si se tiene que implementar en procesos. Como alternativa se puede hacer el procesos en 2 partes. Primero anotando la coincidencias en la matriz por fila, y luego hacer la suma de los valores en las diagonales.

Primero se iniciará la matriz $n * m$ donde n es el largo de la cadena x y m el largo de la cadena y . Luego se agregaría un 1 en las celdas donde coinciden los elementos de ambas cadenas.

	A	C	A	B
A	1	0	1	0
B	0	0	0	1
C	0	1	0	0
A	1	0	1	0
B	0	0	0	1

Siguiendo la lógica de las diagonales del primer algoritmo se suma en cada celda el mayor valor entre la diagonal izquierda superior y la celda izquierda generando una matriz que guarda el tamaño de los subconjuntos encontrados.

	A	C	A	B
--	---	---	---	---

A	1	<u>1</u>	1	1
B	0	<u>1</u>	1	2
C	0	1	1	1
A	1	1	2	<u>2</u>
B	0	1	1	3

En este caso el subconjunto más grande encontrado es de tamaño 3 y siguiendo la diagonal formada encontramos que el subconjunto más grande es “ACAB”

El propósito de este algoritmo es simplificar el paso de mensajes entre procesos. Cada fila se puede procesar y luego el cálculo de las diagonales se puede hacer sumando cada fila procesada a la que le sigue. Esto da oportunidad de usar reducción entre los procesos. Segundo, al ser una fila de una matriz el bloque contiguo de memoria se puede enviar con mas facilidad a través de paso de mensaje. Por último estos bloques contiguos de memoria luego se pueden romper aún más en procesos paralelos donde cada hilo se encarga de un segmento de la cadena.

pseudocodigo:

LCSen2Partes(cadena x, cadena y)

matrizS = matriz tamaño length.x+1 * length.y+1

matrizS = {0}

for i=1 to lenght.x+1

 for j=1 lenght.y +1

 if (elementos match)

 matrizS[i,j] = 1

MayorS =0

IndiceMayor

for i=1 to lenght.x+1

 for j=1 lenght.y +1

 matrizS[i,j] += max(matrizS[i,j-1], matrizS[i-1, j-1]

 if (matriz[i,j]> MayorS)

 MayorS= matriz[i,j]

 IndiceMayor = &matriz[i,j]

#El siguiente proceso busca la subsecuencia más grande.

1) se suma el carácter al cual se conecta al índice mayor

a) Se recorrer la parra arriba de la matriz si el valor es igual al valor mas alto

- b) si es menor se recorre diagonalmente
 - i) cuando se recorre diagonalmente se agrega el caracter conectado a ese índice a la subsecuencia
- c) se termina cuando la celda siguientes son iguales a 0

Referencia para el algoritmo :

<https://youtu.be/Ua0GhsJSIWM?si=0ijepnDucY0hiBqU>